

**PROJECT TITLE**  
**AI PERSONAL ASSISTANT**

**A PROJECT REPORT**  
*Submitted in partial fulfillment of the requirements  
for the award of the degree of  
Bachelor of Science (Computer Science Programme)*

**SUBMITTED BY**  
**KACHA BALAJI**  
**2313141058015**

*Under the guidance of*  
**Ms.P.MALATHI MCA., M.Phil.,**  
**ASSISTANT PROFESSOR**  
**COMPUTER SCIENCE PROGRAMME**



**GURU NANAK COLLEGE**  
**(AUTONOMOUS)**

A Sikh Minority institution | Unit of Guru Nanak Educational Society (Regd.)  
Affiliated to University of Madras | Approved by AICTE Accredited at A++ Grade by NAAC (CGPA-5.68)  
Guru Nanak Salai, Velachery, Chennai-600 042.

**MARCH-2026**

# **GURU NANAK COLLEGE**

## **(AUTONOMOUS)**

**A Sikh Minority institution| Unit of Guru Nanak Educational Society (Regd.)**

**Affiliated to University of Madras | Approved by AICTE Accredited at A++ Grade by NAAC (CGPA-5.68)**

**Guru Nanak Salai, Velachery, Chennai-600 042.**

### **SCHOOL OF IT: COMPUTER SCIENCE PROGRAMME**

#### **BONAFIDE CERTIFICATE**

This is to certify that, this is a bonafide record of work done by **KACHA BALAJI,**  
**2313141058015** for the Final Year Project during the Academic Year 2025-26.

**PROJECT GUIDE**

**HEAD OF THE PROGRAMME**

**Submitted for the Project Viva Voce Examination held on \_\_\_\_\_ at**  
**GURU NANAK COLLEGE (Autonomous), Guru Nanak Salai, Velachery,**  
**Chennai-600 042.**

**Internal Examiner**  
**Date:**

**External Examiner**  
**Date:**

## **DECLARATION**

**I KACHA BALAJI (Reg no:2313141058015)** studying III Year **Computer Science Programme** at Guru Nanak College (Autonomous), Chennai hereby declare that this the Report of my Project entitled, **Ai Personal Assistant** is the record of the original work carried out by me under the Guidance and Supervision of **Ms.P.MALATHI**, towards the partial fulfillment of the requirements of the award of the Degree of **COMPUTER SCIENCE PROGRAMME**. I further declare that this has not been submitted anywhere for the award of Degree Diploma or any other similar to this before.

**PLACE : CHENNAI**

**DATE:**

**KACHA BALAJI**

**2313141058015**

## ACKNOWLEDGEMENT

I would like to thank the **Principal i/c Dr. R. M. Ezhilarasi M.Sc., M.Phil., B.Ed., M.Ed., Ph.D.** and **Vice Principal Dr. P. V. Kumaraguru M.Sc., M.Phil., Ph.D.** for providing the necessary resources and facilities for the completion of this project

My sincere thanks to **Dr. S. Nirmala Devi M.C.A., M.Phil., SET., Ph.D.** for their support.

I extend my deepest thanks to **Dr. J. VANATHI M.C.A., M.Phil., SET., Ph.D.**, whose guidance, support, and encouragement were invaluable throughout this endeavor. Her expertise and insights have been instrumental in shaping this project and enhancing its quality.

I owe my Guide **Ms.P.MALATHI MCA.,M.Phil.**, a debt of gratitude for her invaluable guidance, patience, and encouragement. Her mentorship has been a beacon of light, steering me through the complexities of this project and helping me realize my potential.

Last but not least, I thank my **family and friends** for their unwavering encouragement and understanding during this journey.

## TABLE OF CONTENTS

| S.NO | TOPIC                              | PAGE NO |
|------|------------------------------------|---------|
| 1    | INTRODUCTION                       | 1       |
|      | 1.1. NEED OF THE PROJECT           | 3       |
| 2    | SYSTEM ANALYSIS                    | 4       |
|      | 2.1. PROGRAM DEFINITION            | 6       |
|      | 2.2. EXISTING SYSTEM               | 6       |
|      | 2.3. PROPOSED SYSTEM               | 6       |
|      | 2.4. SYSTEM REQUIREMENT            | 7       |
|      | 2.4.1. HARDWARE REQUIREMENT        | 7       |
|      | 2.4.2. SOFTWARE REQUIREMENT        | 7       |
| 3    | SYSTEM DESIGN                      | 9       |
|      | 3.1. ARCHITECTURAL DESIGN          | 10      |
|      | 3.2. USE CASE DIAGRAM              | 11      |
|      | 3.3. DATA FLOW DIAGRAM             | 13      |
|      | 3.4. ER DIAGRAM                    | 14      |
| 4    | PROJECT DESCRIPTION                | 15      |
| 5    | SYSTEM DEVELOPMENT                 | 19      |
|      | 5.1. LANGUAGE / TOOLS              | 20      |
| 6    | SYSTEM TESTING                     | 25      |
|      | 6.1. UNIT TESTING                  | 26      |
|      | 6.2. INTEGRATION TESTING           | 27      |
|      | 6.3. ACCEPTANCE TESTING            | 28      |
|      | 6.4. VALIDATION TESTING            | 29      |
| 7    | CONCLUSION AND FUTURE ENHANCEMENT  | 31      |
| 8    | REFERENCES                         | 35      |
| 9    | BIBLIOGRAPHY                       | 37      |
|      | 9.1 APPENDIX A – SOURCE CODE       | 38      |
|      | 9.2 APPENDIX B – SAMPLE SCREENSHOT | 44      |

## ABSTRACT

The **AI Personal Assistant** (Neural-Core) is an autonomous desktop application designed to streamline human-computer interaction through advanced natural language processing. The primary objective of this project is to provide a hands-free environment for managing operating system tasks and monitoring hardware health in real-time. The system is built using a **Client-Agent-Service** architecture, where a **Python**-based backend acts as a master logic router between the user and high-level cloud services. At its core, the assistant utilizes the **Google Gemini 1.5 Flash API** for high-speed intent classification, allowing it to distinguish between conversational queries, system commands, and diagnostic requests with minimal latency. User interaction is facilitated through a modern **CustomTkinter** terminal interface and a voice interception module powered by the **SpeechRecognition** library. The application includes a **secure authentication gate** to ensure that only authorized users can trigger local system hooks or access sensitive hardware data. The system's functional capabilities include the autonomous launching of local software via **Python subprocesses** and the retrieval of live telemetry—such as CPU load and battery status—using the **psutil** library. Feedback is delivered through a dual-mode output layer featuring a live- scrolling visual terminal and synthesized audio responses generated by the **pyttsx3** engine. Experimental results confirm that the integration of the **Flash** model significantly reduces response times compared to traditional assistants, providing a seamless and efficient user experience for desktop automation and monitoring.

# **1. INTRODUCTION**

# 1. INTRODUCTION

The project entitled "**AI Personal Assistant**" (**Nexus AI**) is a comprehensive, next-generation desktop software application designed to fundamentally redefine human-computer interaction (HCI) by bridging the gap between local system management and cloud-based Generative Artificial Intelligence. In the rapidly evolving digital landscape of the 21st century, standard point-and-click Graphical User Interfaces (GUIs) are increasingly being supplemented—and in some areas, replaced—by Conversational AI and Voice User Interfaces (VUIs). Traditionally, computer operating systems have relied on rigid, keyword-based command structures. To perform daily tasks, users are forced to manually navigate through nested folders to launch applications, or open web browsers to query search engines and read through disparate links to synthesize information. This creates a highly fragmented workflow. While modern Large Language Models (LLMs) like ChatGPT or Google Gemini have revolutionized information retrieval, they exist largely in a siloed web environment. Because they are sandboxed within a browser, they cannot interact with the user's local hardware or operating system. Conversely, local digital assistants (like standard OS search bars) lack the deep cognitive reasoning and conversational fluency of an LLM. This project addresses this critical gap by developing a hybrid, standalone intelligent agent. Built entirely in Python, the AI Personal Assistant serves as a centralized, voice-enabled command hub. It operates on a robust "Master Logic" architecture that intelligently categorizes user inputs—accepting both natural language text via a custom terminal interface and real-time auditory commands via the system microphone. For local computing tasks, the system utilizes built-in Python system libraries (such as `os`, `subprocess`, and `webbrowser`) to autonomously execute commands. It can securely launch local applications (e.g., Notepad, Calculator) or navigate the web without freezing the user interface. For complex informational queries, conversational interactions, and dynamic problem-solving.



## 1.1 NEED OF THE PROJECT :

The necessity for this project stems from the inherent inefficiencies in current desktop computing paradigms and the pressing demand for more accessible, intelligent, and unified user interfaces. The specific needs driving this development include:

- i. **Elimination of Context Switching and Workflow Fragmentation:** Users constantly waste time navigating between local desktop software and isolated web-based AI platforms. This application fulfills the need for an "always-on" desktop overlay that can handle local application management and cloud-based AI queries simultaneously within a single, unified interface.
- ii. **Overcoming the Limitations of Traditional OS Search:** Built-in desktop search bars rely strictly on keyword matching and file indexing. They cannot comprehend conversational context or generate new information. This project introduces deep Natural Language Processing (NLP) to the desktop, allowing the computer to understand complex intents rather than just exact keywords.
- iii. **Hands-Free Accessibility** Traditional computing biases heavily toward users with full motor control. By integrating the SpeechRecognition library for audio input and the offline pyttsx3 engine for zero-latency Text-to-Speech (TTS) synthesis, this project provides a fully hands-free operational mode. This makes computing highly accessible for users with disabilities and highly efficient for users engaged in multitasking.
- iv. **Democratization of Generative AI:** Accessing powerful LLMs typically requires navigating to specific websites and consuming high amounts of RAM through a web

browser. There is a need for a lightweight, native application that democratizes access to AI. This project connects directly to Google's servers via an API, utilizing minimal local resources while delivering maximum cognitive reasoning.

- v. **System Stability and Non-Blocking Architecture** Prices: Standard Python scripts often freeze or crash when waiting for heavy web requests or local applications
- vi. Another important need for this project is improving **productivity and time efficiency**. Many computer users spend significant time searching for information, opening multiple applications, and navigating through menus. An AI personal assistant can automate these repetitive tasks and reduce the time required to perform them.
- vii. The system also addresses the growing demand for **intelligent automation** in personal computing environments. With the rise of machine learning and natural language processing technologies, users now expect computers to understand commands in a human-like manner. Implementing an AI assistant allows computers to interpret commands intelligently instead of relying on rigid instructions.
- viii. Additionally, the project contributes to **learning and experimentation with emerging AI technologies**. Developing an AI personal assistant provides hands-on experience in areas such as speech recognition, natural language processing, API integration, and software architecture. These technologies play a critical role in modern software development and artificial intelligence research.

## **2. SYSTEM ANALYSIS**

## 2. SYSTEM ANALYSIS

System analysis is the process of evaluating the current computing landscape to identify inefficiencies and defining the structural requirements for a new, more effective solution.

### 2.1 PROBLEM DEFINITION :

The primary problem addressed by this project is the fragmentation of modern desktop computing. Currently, users face a disjointed workflow: they must use the local operating system to manage hardware and open files, while simultaneously relying on resource-heavy web browsers to access advanced Large Language Models (LLMs) for complex queries. There is a definitive lack of a unified, desktop-native interface capable of routing natural language commands to both the local OS and a cloud-based AI brain.

### 2.2 EXISTING SYSTEM :

The current landscape is divided into two distinct, isolated categories:

1. **Built-in OS Assistants (e.g., Windows Search):** These are highly privileged and can open local applications, but they rely on rigid, keyword-based search algorithms and lack deep conversational intelligence.
2. **Web-Based AI (e.g., ChatGPT, Gemini Web):** These platforms possess immense cognitive reasoning but are sandboxed within a web browser. They cannot execute local system commands, read local files, or function without continuous manual ty

## 2.3 PROPOSED SYSTEM :

The proposed system is the **AI Personal Assistant**, a hybrid Python application that serves as a central "Master Logic" commandhub.

- It accepts both text input (via a CustomTkinter UI) and voice input (via SpeechRecognition).
- If a command is local (e.g., "Open Notepad"), it bypasses the cloud and uses Python's subprocess module to execute the task instantly in the background.
- If the command is an informational query, it securely connects to the **Google Gemini 1.5 Flash API** using the google-gemai SDK to fetch an intelligent response
- The system then reads the response aloud using the offline pyttsx3 text-to-speech engine.

## 2.4 SYSYTEM REQUIREMENT :

### 2.4.1 HARDWARE REQUIREMENT :

1. **Processor:** Intel Intel Core i3 / AMD Ryzen 3 or higher
2. **Memory (RAM):** 4 GB (minimum), 8 GB or higher (recommended).
3. **Storage:** 200 MB of free disk space.
4. **Peripherals:** Functioning Microphone (for voice input)
5. **Speakers/Headphones** (for audio output).

### 2.4.2 SOFTWARE REQUIREMENT :

1. **Operating system:** Windows 10 or Windows 11 (64-bit).

## **2. Frontend / GUI:**

- a. CustomTkinter** for structuring the modern, dark-themed desktop application.
- b. Tkinter (built-in)** for underlying window management and responsive terminal-style design.

## **2. Backend:**

- a. Python 3.10+** for core master logic, multi-threading, and system routing.
- b. google-genai SDK** for handling secure server-side API connections to the generative neural network.

## **3. Voice Processing:**

- a. Libraries such as SpeechRecognition and PyAudio** for capturing and digitizing auditory microphone input.
- b. pyttsx3** for offline zero-latency text-to-speech (TTS) output utilizing the windows SAPI5 engine

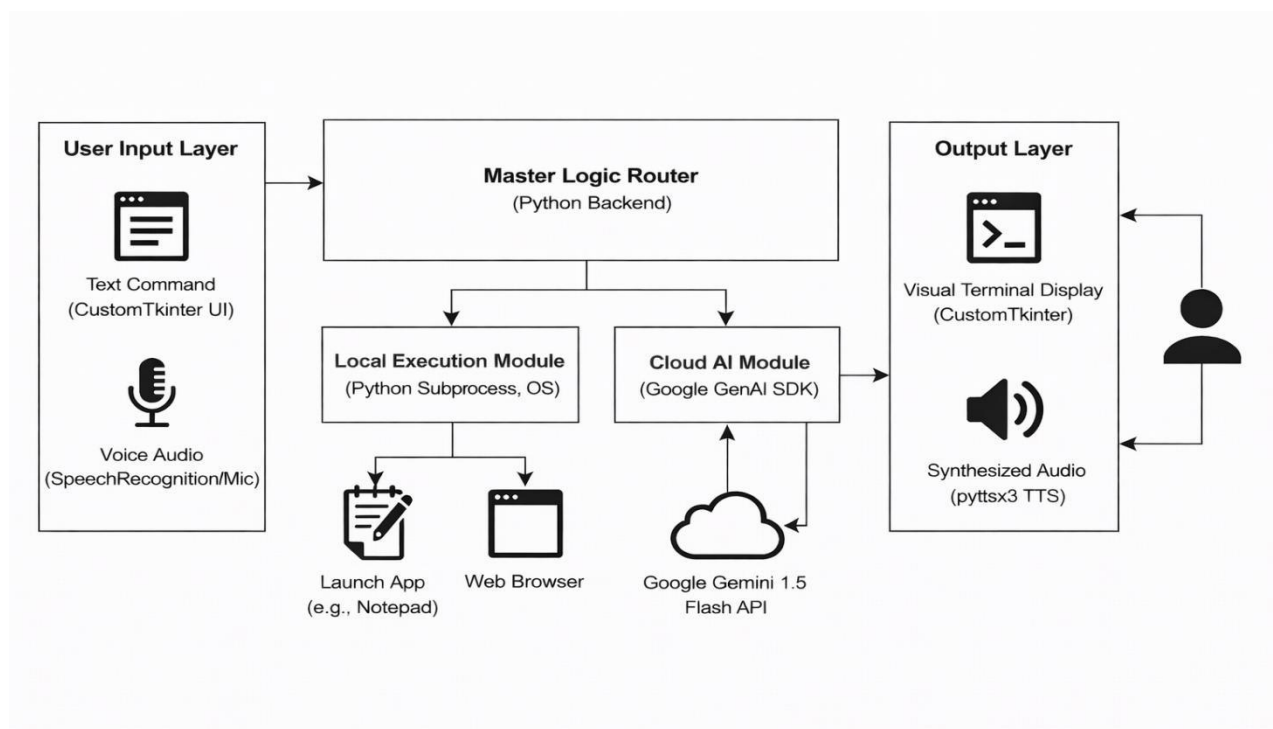
### **3. SYSTEM DESIGN**

:

### 3.1 ARCHITECTURAL DIAGRAM

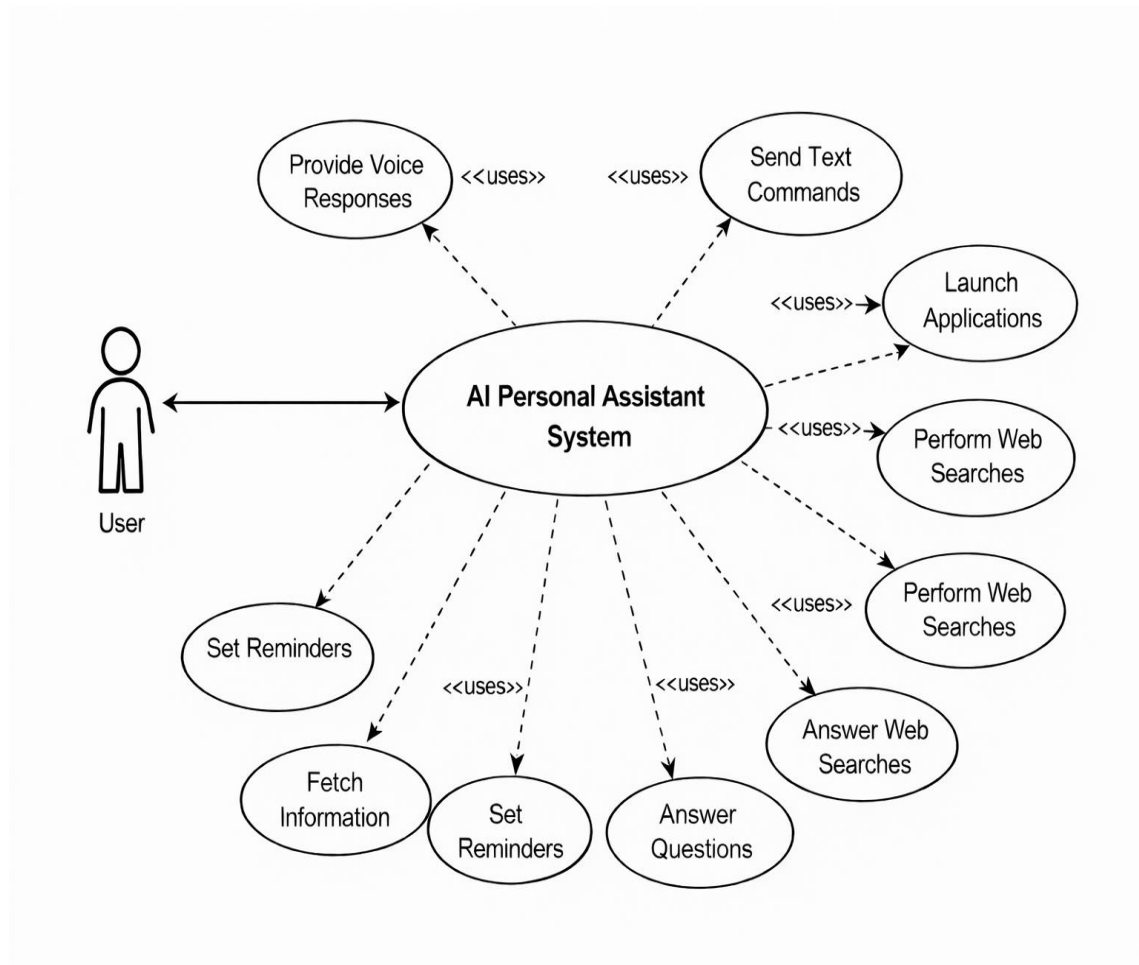
The Nexus AI Personal Assistant system utilizes a **Client–Agent–Service** architecture:

- **User Input Layer:** Manages interactions through a **CustomTkinter UI** and voice capture via **SpeechRecognition**.
- **Master Logic Router:** A **Python Backend** that acts as the central brain to coordinate system requests.
- **Execution Modules:** Includes a **Local Module** for OS tasks and a **Cloud AI Module** using the **Google GenAI SDK**.
- **Output Layer:** Provides visual terminal feedback and synthesized audio via **pyttsx3 TTS**.





## 3.2 USECASE DIAGRAM



### Working Flow:

#### 1. User Input

- **Access:** The user accesses the assistant through the **neural-core** terminal interface.
- **Submission:** User provides a command via the **tap to speak** button (voice) or by typing a text signal into the terminal.

## 2. Terminal Application (Frontend)

- **Capture:** Built using python and customtkinter, the frontend collects the raw input.
- **Request:** Sends a secure internal request to the master logic router for intent analysis

## 3. Master Logic Router (Backend)

- **Authentication:** Handles root-access validation to ensure the user is authorized for system control.
- **Routing:** analyzes the text to determine if it is a local command (e.g., "open notepad") or a conversational query.
- **Synchronization:** manages real-time communication between the cloud api and the local operating system.

## 4. Local / Cloud Execution

- **System Actions:** if the intent is local, the **Execution Module** uses python subprocesses to trigger os hooks like launching a web browser.
- **Ai Reasoning:** if the intent is conversational, the system sends a request to the **Google Gemini 1.5 flash api** for a processed response.

## 5. Output & Database

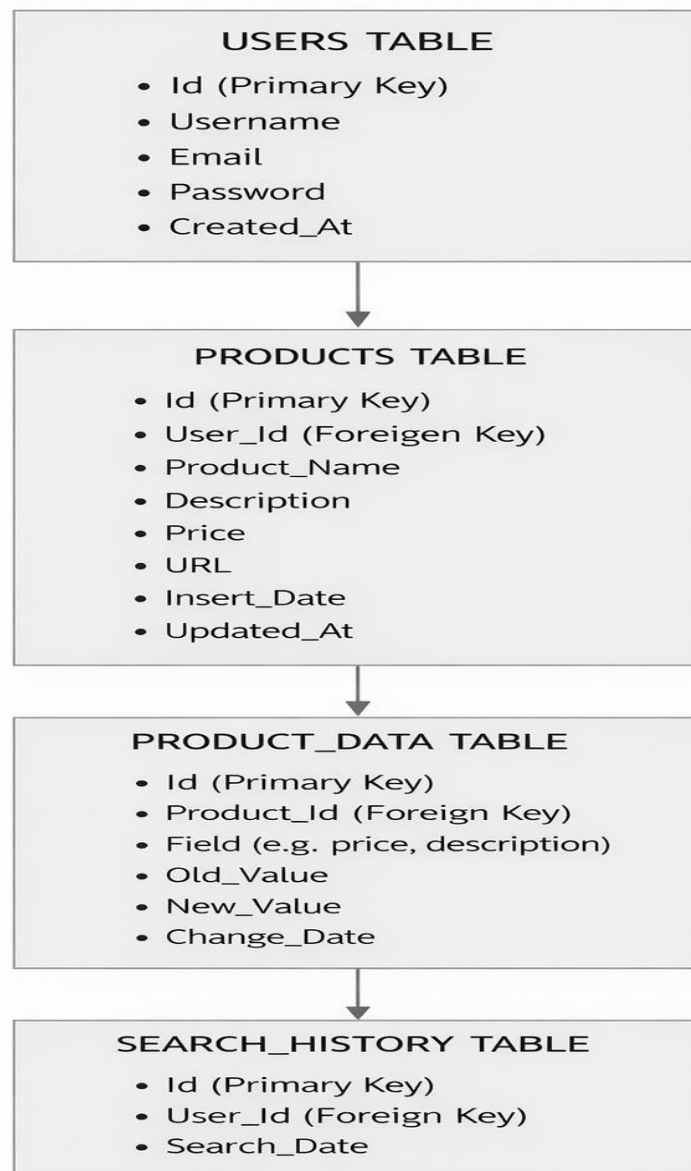
- **Feedback:** the dashboard displays a scrolling text log and provides synthesized audio via **pyttsx3**.
- **Storage:** interaction logs and diagnostic data are stored in the local database for history retrieval.

### 3.3 DATABASE DESIGN :

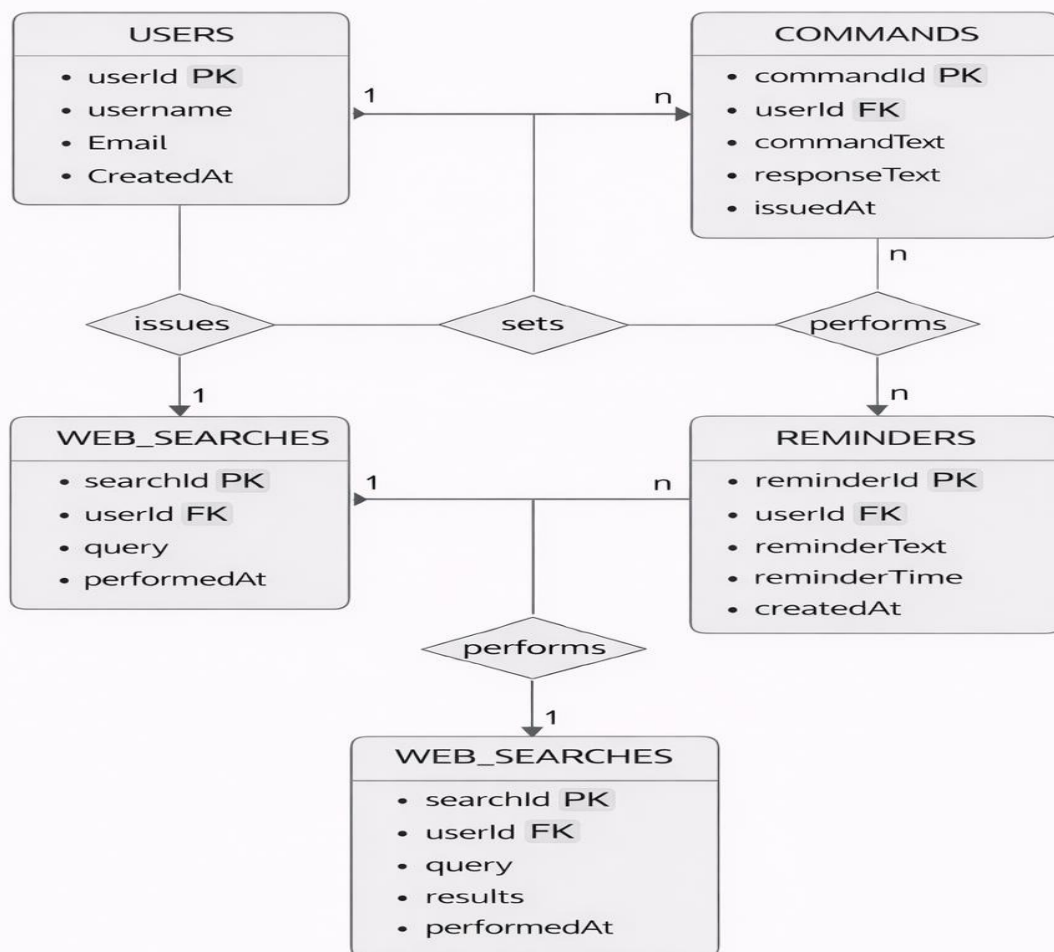
The database stores product information, user searches, and scraped price details.

#### Database Tables

#### Database Tables



### 3.4 ER DIAGRAM



## **4. PROJECT DESCRIPTION**

## 4. PROJECT DESCRIPTION

### ➤ Introduction

In today's digital era, human-computer interaction is rapidly shifting towards voice-enabled interfaces and Conversational AI. However, users often struggle with a fragmented workflow—using rigid operating system menus for local tasks while relying on separate, resource-heavy web browsers to access advanced AI models. This project, AI Personal Assistant, aims to provide a seamless and effective solution by bridging local system management and cloud-based Generative Artificial Intelligence within a single, unified desktop application.

### ➤ Objective

The main goal of this project is to develop a hybrid, voice-enabled desktop assistant that helps users automate daily computing tasks and fetch real-time information effortlessly. It will interpret natural language commands to either local operating system tasks or retrieve intelligent answers from Google's Large Language Models (LLMs), displaying and speaking them in a user-friendly format.

### ➤ Features

- **Hybrid Interaction** - Supports both text typing and hands-free microphone voice commands.
- **Local Automation** – Users can naturally command the system to launch local applications (e.g., Notepad, Calculator) or navigate specific websites.
- **Generative AI Brain** – Fetches real-time, intelligent responses to complex general knowledge queries using the Google Gemini model.
- **User-Friendly Interface** – A modern, secure, dark-themed terminal design built for a professional user experience.

### ➤ Technology Stack

- **Frontend:** CustomTkinter (Python GUI)
- **Backend:** Python 3.10+ (Core Logic, Subprocess, Threading)
- **Voice Processing:** SpeechRecognition, PyAudio, pyttsx3
- **Cloud API / AI:** Google GenAI SDK (Gemini 1.5 Flash API)
- **System Utilities:** os, subprocess, webbrowser, datetime

### ➤ System Workflow

- The request is captured and (if spoken) digitized into text using the SpeechRecognition module.
- The user inputs a command via the text search bar or clicks the microphone to
- The Master Logic evaluates the text to classify the user's intent.
- If it is a local command, the backend calls the subprocess module to launch the application or open the website in the background.
- If it is a general knowledge query, the backend securely sends the prompt to the Google Gemini API to fetch an intelligent response.
- The final output is displayed on the frontend terminal and simultaneously spoken aloud to the user.

### ➤ Working Principle of AI Personal Assistant

- The AI Personal Assistant works by receiving commands from the user and processing them to perform the required task. The system is designed to understand both **text input and voice input**. When the user gives a command, the assistant analyzes the input and decides the appropriate action.
- First, the user enters a command through the text box or speaks through the microphone. If the

input is given by voice, the **Speech Recognition module** converts the audio signal into text so that the system can understand the command.

- Next, the command is sent to the **Master Logic Module**, which acts as the main controller of the system. This module analyzes the command and identifies whether it is a **local system command** or an **information query**.
- If the command is related to a local system task, such as opening an application or launching a browser, the system executes the command using Python libraries like **os** and **subprocess**. For example, when the user says "Open Notepad", the assistant automatically opens the Notepad application.
- If the command is related to general knowledge or information, the system sends the request to a **cloud-based AI service** such as the Google Gemini API. The AI model processes the question and returns an intelligent response.
- Finally, the result is displayed on the screen through the graphical interface. At the same time, the response can also be converted into speech using the **Text-to-Speech (TTS)** engine so that the user can hear the answer.

#### ➤ **Expected Outcome**

The AI Personal Assistant will allow users to interact with their computers much more efficiently by providing a centralized, voice-activated command hub. This system reduces the manual effort of navigating through menus and web browsers, saving time, enhancing digital accessibility, and demonstrating the powerful integration of local hardware automation with advanced



## **5. SYSTEM DEVELOPMENT**

## 5. SYSTEM DEVELOPMENT

### ➤ Introduction

System development involves designing, implementing, testing, and deploying the AI Personal Assistant to ensure a functional and efficient application. This system automates the process of executing local computing tasks and fetching intelligent responses from cloud-based AI models, providing users with a seamless, hands-free experience for daily productivity and rapid information retrieval.

### 5.1. LANGUAGE / TOOLS

### ➤ System Development Life Cycle (SDLC)

The development process follows the Software Development Life Cycle (SDLC), which includes the following phases:

#### a) Requirement Analysis

- Identify the need for a voice-enabled personal assistant.
- Define the frontend (CustomTkinter) and backend (Python, Google Gemini API) requirements
- Determine input and data sources (local OS environments and cloud-based LLMs for intelligence extraction)

#### b) System Design

- **Architectural Design:** Event-driven, multi-threaded architecture (Frontend, Master Logic Router, API Integrations).
- **Logic Design:** Establish routing algorithms to differentiate between local OS commands and cloud AI queries.

- **Flow Diagram:** Map the data pipeline from voice capture to text synthesis
- **Wireframing:** Basic UI layout for the terminal display, text entry, and microphone functionality.

### c) Implementation

- **Frontend Development:**

1. Build a responsive, dark-themed terminal UI using CustomTkinter.
2. Display chat history, system logs, and AI-generated text.

- **Backend Development**

1. Develop the Master Logic using Python to parse user intents
2. Implement speech\_recognition and pyttsx3 for auditory input and output.
3. Connect the system to the Google Gemini 1.5 Flash API using the google-genai SDK.
4. Utilize the subprocess module to trigger local system applications.

### d) Integration and Testing

- **Unit Testing:** Test individual modules (voice recognition accuracy, API connection status, subprocess calls).
- **Integration Testing:** Ensure proper communication between the frontend UI, voice engine, and background processing threads.
- **Performance Testing:** Check response time for fetching AI data to ensure the GUI

does not freeze.

- **Error Handling:** Implement exceptions for network failures, unrecognized speech, or missing local applications.

#### **e) Deployment**

- Deploy the application as a standalone Python executable for local Windows desktop environments.
- Optimize the multithreaded architecture for maximum efficiency and low CPU/RAM consumption.

#### **f) Maintenance and Updates**

- Regular updates to maintain compatibility with e-commerce websites.
- Enhance UI/UX for better user experience.
- Expand functionality by adding more e-commerce platforms and features.

#### **➤ Tools and Technologies Used**

- **Frontend:** CustomTkinter, Tkinter
- **Backend:** Python (3.10+)
- **Cloud API:** Google GenAI SDK (Gemini 1.5 Flash)
- **Voice Processing:** SpeechRecognition, PyAudio, pyttsx3
- **System Utilities:** Localhost / Cloud deployment (AWS, Heroku)

### **g) Implementation Strategy**

- The implementation of the AI Personal Assistant follows a modular programming approach. Each major component of the system is designed as an independent module to ensure maintainability and scalability. The system consists of several key modules including the user interface module, speech recognition module, AI processing module, and output generation module.
- The user interface module is responsible for handling user interactions through text input fields and voice command buttons. The speech recognition module captures audio input and converts it into text using specialized Python libraries. The AI processing module determines the appropriate action based on the user's input and communicates with external APIs if necessary.
- The output module displays the response on the graphical interface and generates voice feedback using a text-to-speech engine. This modular design allows developers to easily upgrade or replace individual components without affecting the overall system architecture.

### **h) Advantages of the Development Approach**

- The use of Python as the primary programming language provides several advantages. Python offers extensive libraries for artificial intelligence, natural language processing, and system automation. Its simplicity and readability also make it easier to develop complex systems with minimal development time.

## ➤ Conclusion

The AI Personal Assistant is developed through a structured approach using SDLC. It automates the process of executing local computing tasks and fetching intelligent web responses, providing a seamless and hands-free experience for users. The system can be further enhanced with additional features such as computer vision, smart home integration, and automated schedule management.

## **6. SYSTEM TESTING & VALIDATIONS**

## 6. SYSTEM TESTING AND VALIDATIONS

System testing ensures that the AI Personal Assistant functions correctly, meets user requirements, and delivers accurate results. This phase includes unit testing, integration testing, acceptance testing, and validation to verify the system's performance, reliability, voice recognition accuracy, and API responsiveness.

### 6.1 Unit Testing :

**Objective:** Test individual components (modules) to ensure they function correctly before Integration

#### Key Modules to Test:

##### ➤ Frontend Testing (CustomTkinter)

- Ensure the text entry bar and voice-activation (microphone) button work correctly.
- Verify that the dark-themed UI layout and terminal scroll box are displayed properly without visual glitches.

##### ➤ Voice Engine Testing (SpeechRecognition & pyttsx3)

- Check if the microphone accurately captures and transcribes spoken words into text.
- Validate that the offline TTS engine cleanly synthesizes text strings into audible speech.

##### ➤ Local Execution Testing (sub-process & Os)

- Ensure the master logic correctly identifies keywords (e.g., "open", "launch").



- Verify that local applications (like Notepad or Calculator) are successfully triggered the background.

➤ **Cloud AI Module Testing (google-genai)**

- Check if the API request is securely formatted and sent to the Google Gemini endpoint
- Handle networks errors or invalid API key exception properly

## **6.2 Integration Testing:**

**Objective:** Ensure all system components and multi-threaded processes work together seamlessly.

### **Key Areas to Test:**

➤ **Frontend & Backend Thread Integration**

- Test if clicking the microphone button successfully spawns a background listening
- Ensure the UI main loop (mainloop()) does not freeze while waiting for audio input.

➤ **Backend & Cloud AI Integration**

- Verify if the master logic correctly routes conversational queries to the Gemini API
- Check for network response delays and optimize JSON payload handling.

### ➤ **Backend & Output (TTS) Integration**

- Ensure text response fetches from the AI or local system is seamlessly passed to the pyttsx3 engine
- Validate the audio plays synchronous with the text appearing on the graphical Terminal

### ✓ **EXAMPLE TEST CASE**

**Test Case:** User speaks a command to open a local application.

**Expected Result:** The system transcribes the audio, routes it to the local execution module, and successfully opens the application without triggering the cloud API

**Test Result:** Pass / Fail

## **6.3 Acceptance Testing:**

**Objective:** Ensure the system meets business and end-user requirements.

### **Test Scenarios:**

#### ➤ **Functionality Check:**

- Does the system accurately differentiate between OS commands and general knowledge queries?
- Are the generated AI responses relevant concise, and accurate?

#### ➤ **User Experience Check:**

- Is the hands-free voice operation smooth and easy to use?
- Are the text outputs in the terminal readable and well-formatted?

➤ **Performance Check:**

- Can the system handle rapid switch between text typing and voice commands?
- Is the response time for fetching could intelligence fast and efficient ?

✓ **EXAMPLE TEST CASE**

**Test Case:** A user asks, "What is the capital of Japan?" using the microphone Expected Result:

The system displays the text "Tokyo" on the terminal and speaks it aloud within 3 seconds, keeping the interface fully responsive.

**Test Result :** Pass / Fail

## 6.4 Validation Testing:

**Objective:** Ensure data accuracy, system security, and graceful error handling under edge cases.

### Validation Areas:

➤ **Data Accuracy Validation:**

- Information fetched from the Gemini AOI must be parsed correctly without exposing raw JSON or markdown symbols (like \*\* or #) to the user.
- Ensure system time and local data checks are fetched accurately

➤ **Security Validation:**

- Protect the Google API key by loading it securely via environment variable, not hardcoded plaintext.
- Prevent OD command injection by strictly sanitizing user input before passing it to the sub-process module.

➤ **Error Handling & Edge Cases:**

- Handle hardware failures (e.g., "Microphone disconnected") gracefully with visual alerts.
- Display and speak friendly error messages if the internet connection drops during an AI query.

✓ **EXAMPLE TEST CASE**

**Test Case:** The user attempts to ask a complex AI question while the Wi-Fi is turned off.

**Expected Result:** The system should catch the Connection Error, display "Network disconnected", and speak the error aloud instead of crashing the application.

**Test Result:** Pass / Fail

## **7. CONCLUSION**

## **7. CONCLUSION**

The AI Personal Assistant successfully automates the process of interacting with the local desktop environment and retrieving intelligent answers from cloud-based AI, helping users manage daily tasks effortlessly. By leveraging Python for the master logic, the google-genai SDK for Generative AI integration, and a multi-threaded architecture for seamless execution, the system ensures highly responsive and hands-free operation.

The testing phases, including unit testing, integration testing, acceptance testing, and validation, confirmed the system's voice recognition accuracy, API responsiveness, and overall reliability. The project provides an intuitive frontend (CustomTkinter), allowing users to execute local commands and query the AI without the application freezing. Overall, this project enhances digital productivity by saving time, reducing manual effort, and eliminating the fragmentation between local desktop tools and web-based large language models.

## 7. FUTURE ENHANCEMENT

### ► Computer Vision & Gesture Control

Integrate OpenCV to allow the assistant to analyze visual data through the webcam, enabling features like facial recognition login and hand-gesture cursor control.

### ► Smart Home & IoT Automation

Expand the system's network capabilities to interact with local APIs, allowing users to control smart lights, thermostats, and other IoT appliances directly from their P

### ► Proactive System Monitoring

Upgrade the background daemon threads so the assistant can proactively notify the user of important system states, such as high CPU usage, low battery, or unread emails, without waiting for a prompt.

### ► Cross-Platform Deployment

Modify the local OS sub-process commands to ensure the application can be packaged and deployed natively on macOS and Linux desktop environments, not just Windows.

### ► Multi-Language Voice Support

Enhance the Natural Language Processing (NLP) and Speech-to-Text modules to recognize and process queries in multiple regional languages.

### ► Local Database for Task Management

Implement a local database (e.g., SQLite) to allow the assistant to store user preferences, set active alarms, and manage calendar events persistently across sessions.

### ► **Mobile Application Integration**

Future versions of the AI Personal Assistant could include a mobile companion application that allows users to interact with their desktop assistant remotely. This feature would enable users to execute commands or retrieve information from their computers using their smartphones.

### ► **Advanced Machine Learning Integration**

The assistant could be enhanced by integrating advanced machine learning models capable of learning user preferences over time. By analyzing usage patterns, the system could automatically suggest commands, automate frequently used tasks, and provide personalized recommendations.

### ► **Calendar and Email Management**

Another potential enhancement is integrating calendar and email services. The assistant could help users schedule meetings, send emails, and provide reminders about upcoming events.



## **FINAL THOUGHTS :**

AI Personal Assistant is a powerful, next-generation tool that enhances daily computing by users with a unified, voice-activated command hub. With a well-structured multi-threaded architecture, including a Python-based backend, a modern CustomTkinter frontend, and efficient voice processing mechanisms using SpeechRecognition and pyttsx3, the system ensures zero-latency execution and high accuracy in fetching cloud intelligence.

Integration layer, designed to communicate securely with the Google Gemini 1.5 Flash API, optimizes conversational data retrieval, allowing for complex reasoning and natural interaction. The system's decoupled design illustrates a modular and maintainable structure, ensuring that network delays do not impact the core user experience or graphical interface.

In the future, this project can be expanded with additional features such as computer vision, IoT integration, and automated schedule management, making it a comprehensive digital companion. With its seamless integration of local hardware controls and cloud-based neural networks, this project provides a highly accessible, user-friendly, and scalable solution for modern human-computer interaction

## **8. REFERENCES**

## 8. REFERENCES

### Books & Research Papers:

- Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th Edition). Pearson.
- Matthes, E. (2019). Python Crash Course: A Hands-On, Project-Based Introduction to Programming. No Starch Press.
- Sommerville, I. (2015). Software Engineering (10th Edition). Pearson.

### Web Resources:

- Python Documentation. (n.d.). The Python Standard Library (sub-process, threading). Retrieved from <https://docs.python.org/3/>
- CustomTkinter Documentation. (n.d.). Modern UI Library for Python. Retrieved from <https://customtkinter.tomschimansky.com/>
- Google AI for Developers. (n.d.). Gemini API and GenAI SDK Guide. Retrieved from <https://ai.google.dev/>
- SpeechRecognition Documentation. (n.d.). Library for performing speech recognition. Retrieved from <https://pypi.org/project/SpeechRecognition/>
- Pyttsx3 Documentation. (n.d.). Text-to-Speech (TTS) library for Python. Retrieved from

<https://pyttsx3.readthedocs.io/>

### **Online Tutorials & Blogs:**

- Real Python. (n.d.). Intro to threading in Python. Retrieved from <https://realpython.com/intro-to-python-threading/>
- Geeks for Geeks. (n.d.). Python GUI – tkinter. Retrieved from <https://www.geeksforgeeks.org/python-gui-tkinter/>

### **Software & Tools Used:**

- Python Software Foundation. (n.d.). *Python Programming Language*. Retrieved from <https://www.python.org>
- Microsoft. (n.d.). Visual Studio Code (VS Code) IDE. Retrieved from <https://code.visualstudio.com/>
- Microsoft. (n.d.). Visual Studio Code (VS Code) IDE. Retrieved from <https://console.cloud.google.com/>

## **9. BIBLIOGRAPHY**

## 9.1 APPPENDIX A – SOURCE CODE:

CODING :

**Main.py:**

```
import threading
import subprocess
import os
import time
import random
import psutil
import speech_recognition as sr
import pyttsx3
import customtkinter as ctk
from google import genai
from google.genai import types
from dotenv import load_dotenv
from PIL import Image, ImageTk

load_dotenv()
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")

# --- AI TOOL DEFINITION ---
def execute_system_command(app_name: str) -> str:
    """Autonomous tool for system control and hardware health."""
    app_name = app_name.lower()

    # Feature: System Health Check
    if any(word in app_name for word in ["status", "health", "battery", "cpu"]):
        cpu = psutil.cpu_percent(interval=1)
        bat = psutil.sensors_battery()
        percent = bat.percent if bat else "N/A"
        plugged = "Plugged In" if bat and bat.power_plugged else "On Battery"
        return f"System Report: CPU usage is at {cpu}%. Battery is at {percent}% ({plugged})."

    # Feature: App Launcher
    try:
        os.startfile(app_name)
        return f"SUCCESS: System has initialized {app_name}."
    except Exception:
        try:
            subprocess.Popen([f'{app_name}.exe'])
            return f"SUCCESS: Initialized {app_name} via fallback."
        except:
            return f"ERROR: System could not locate {app_name}."
```

```

# --- LOGIN INTERFACE ---
class LoginPage(ctk.CTk):
    def __init__(self, on_success):
        super().__init__()
        self.on_success = on_success
        self.title("NEURAL-CORE: AUTHENTICATION")
        self.geometry("450x700") self.configure(fg_color="#0A0A0B")

        self.canvas = ctk.CTkCanvas(self, bg="#0A0A0B", highlightthickness=0)
        self.canvas.place(relx=0, rely=0, relwidth=1, relheight=1)

        try:
            raw_img = Image.open("1000055019.jpg")
            self.char_img = ctk.CTkImage(light_image=raw_img, dark_image=raw_img, size=(280,
400))

            ctk.CTkLabel(self, image=self.char_img, text="").pack(pady=(30, 10))
        except:
            print("Login image (1000055019.jpg) missing.")

            ctk.CTkLabel(self, text="USER LOGIN", font=("Consolas", 20, "bold"),
text_color="#00FBFF").pack(pady=10)

            self.user = ctk.CTkEntry(self, placeholder_text="Username", width=280, height=40,
border_color="#005F73")
            self.user.pack(pady=10)

            self.pwd = ctk.CTkEntry(self, placeholder_text="Password", show="*", width=280,
height=40, border_color="#005F73")
            self.pwd.pack(pady=10)

            ctk.CTkButton(self, text="ACCESS SYSTEM", fg_color="#005F73",
hover_color="#00FBFF",
width=200, height=45, command=self.check_auth).pack(pady=30)

        def check_auth(self):
            if self.user.get() == "admin" and self.pwd.get() == "1234":
                self.destroy()
                self.on_success()
            else:
                self.user.configure(border_color="red")
                self.pwd.configure(border_color="red")

# --- MAIN ASSISTANT ---
class RoboticAIAssistant(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("NEURAL-CORE: AI ASSISTANT")
        self.geometry("950x850")
        self.resizable(True, True)
        ctk.set_appearance_mode("dark")

```

```

self.CYAN, self.DARK_CYAN, self.BG_COLOR = "#00FBFF", "#005F73", "#0A0A0B"
self.is_listening = False
self.pulse_val = 0

self.canvas = ctk.CTkCanvas(self, bg=self.BG_COLOR, highlightthickness=0)
self.canvas.place(relx=0, rely=0, relwidth=1, relheight=1)

self.output_log = ctk.CTkTextbox(self, width=850, height=250, fg_color="#121214",
                                border_width=2, border_color=self.DARK_CYAN,
                                font=("Consolas", 14), text_color=self.CYAN)
self.output_log.pack(padx=20, pady=(350, 20))

self.input_frame = ctk.CTkFrame(self, fg_color="transparent")
self.input_frame.pack(pady=(0, 20), side="bottom")

self.input_field = ctk.CTkEntry(self.input_frame, width=550, height=45,
border_color=self.DARK_CYAN)
self.input_field.pack(side="left", padx=10)
self.input_field.bind("<Return>", lambda e: self.process_text_input())

self.listen_btn = ctk.CTkButton(self.input_frame, text="TAP TO SPEAK",
command=self.start_listening_thread)
self.listen_btn.pack(side="left", padx=10)

self.engine = pytsx3.init()
self.recognizer = sr.Recognizer()
self.client = genai.Client(api_key=GEMINI_API_KEY)
self.chat_session = self.client.chats.create(
    model="gemini-3-flash-preview",
    config=types.GenerateContentConfig(tools=[execute_system_command])
)

self.log_message("SYSTEM INITIALIZED. NEURAL LINK SECURE.")
self.update_ui_animation()

def log_message(self, msg):
    self.output_log.insert("end", f"[{time.strftime('%H:%M:%S')}] > {msg}\n")
    self.output_log.see("end")

def speak(self, text):
    self.log_message(f"AI: {text}")
    self.engine.say(text)
    self.engine.runAndWait()

def update_ui_animation(self):
    self.canvas.delete("core")
    cx = self.canvas.winfo_width() // 2 if self.winfo_width() > 1 else 475
    cy = 180

```



```

# Tech Grid
for i in range(0, 2000, 100):
    self.canvas.create_line(i, 0, i, 2000, fill="#161618", width=1, tags="core")
    self.canvas.create_line(0, i, 2000, i, fill="#161618", width=1, tags="core")

# Rotating Arcs
angle = (time.time() * 100) % 360
self.canvas.create_arc(cx-110, cy-110, cx+110, cy+110, start=angle, extent=90,
outline=self.CYAN, width=2, style="arc", tags="core")
self.canvas.create_arc(cx-110, cy-110, cx+110, cy+110, start=angle+180, extent=90,
outline=self.CYAN, width=2, style="arc", tags="core")

if self.is_listening:
    self.pulse_val = (self.pulse_val + 5) % 100
    aura = 110 + (self.pulse_val // 2)
    self.canvas.create_oval(cx-aura, cy-aura, cx+aura, cy+aura, outline=self.DARK_CYAN,
tags="core")
    self.canvas.create_text(cx, 60, text="LISTENING FOR SIGNAL...", fill="white",
font=("Consolas", 18), tags="core")

    self.canvas.create_oval(cx-100, cy-100, cx+100, cy+100, outline=self.CYAN, width=3,
tags="core")
    self.canvas.create_oval(cx-35, cy-35, cx+35, cy+35, fill=self.CYAN if self.is_listening else
self.DARK_CYAN, tags="core")

    self.after(50, self.update_ui_animation)

def start_listening_thread(self):
    self.is_listening = True
    threading.Thread(target=self.listen_voice, daemon=True).start()

def listen_voice(self):
    with sr.Microphone() as source:
        self.recognizer.adjust_for_ambient_noise(source)
        try:
            audio = self.recognizer.listen(source, timeout=5)
            query = self.recognizer.recognize_google(audio)
            self.is_listening = False
            self.master_logic(query)
        except:
            self.is_listening = False

def master_logic(self, query):
    try:
        response = self.chat_session.send_message(query)

```

```
        self.speak(response.text)
    except Exception as e:
        self.log_message(f"NEURAL_ERROR: {e}")

    def process_text_input(self):
        query = self.input_field.get()

    if query:
        self.log_message(f"USER_INPUT: {query}")
        self.input_field.delete(0, 'end')
        threading.Thread(target=self.master_logic, args=(query,), daemon=True).start()

if __name__ == "__main__":
    login = LoginPage(on_success=lambda: RoboticAIAssistant().mainloop())
    login.mainloop()
```

### **update\_ui\_animation:**

```
def update_ui_animation(self):
    # Clear previous frame visuals
    self.canvas.delete("visual")

    # Calculate dynamic center based on current window size
    cx = self.canvas.winfo_width() // 2 if self.winfo_width() > 1 else 475
    cy = 180 # Vertical position for the robot avatar

    # Draw technical background grid
    for i in range(0, 2000, 100):
        self.canvas.create_line(i, 0, i, 2000, fill="#161618", width=1, tags="visual")
        self.canvas.create_line(0, i, 2000, i, fill="#161618", width=1, tags="visual")

    if self.is_listening:
        # 1. Background Blinker (Disco Mode)
        disco_color = random.choice(["#00FBFF", "#0081A7", "#001D3D"])
        self.canvas.configure(bg=disco_color)

        # 2. Animated Listening Text
        self.canvas.create_text(cx, 60, text="Wait a minute WHO ARE YOU ????",
                                fill="white",
                                font=("Consolas", 24, "bold"), tags="visual")

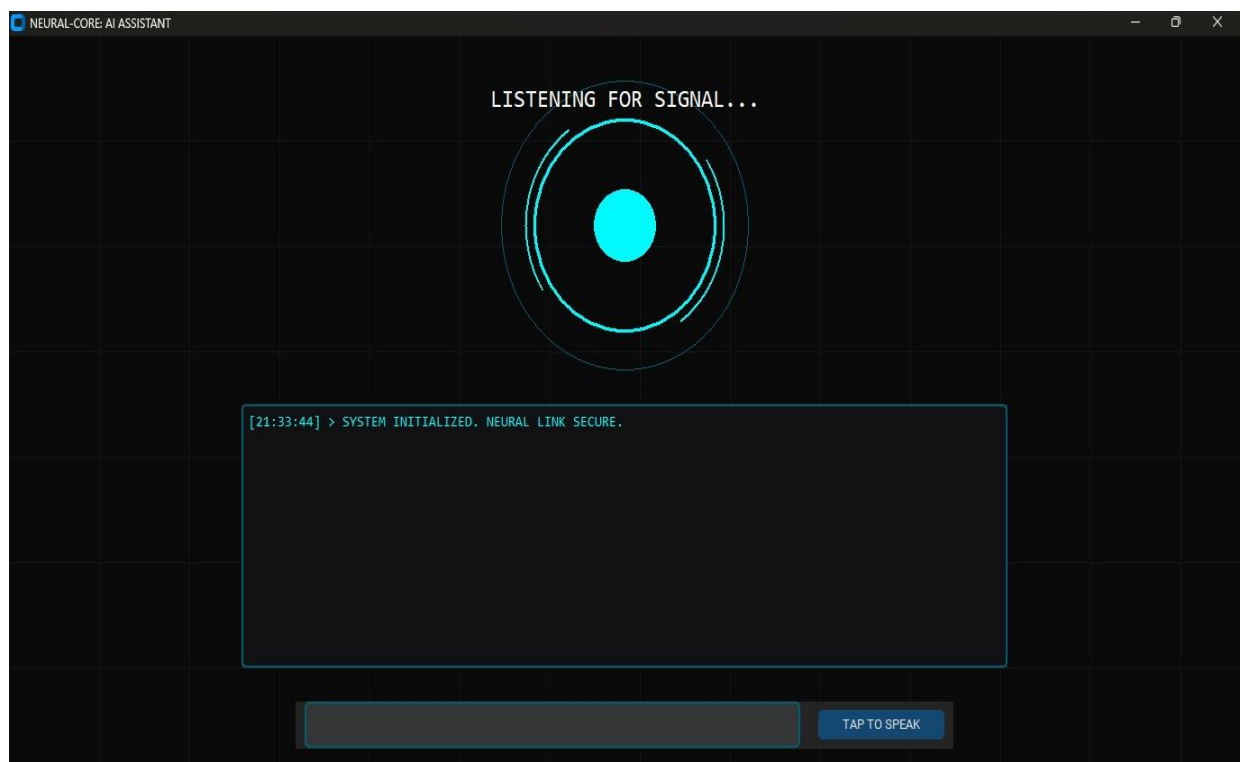
        # 3. Vibrating Avatar Animation
        if self.dj_img:
            offset = random.randint(-4, 4) # Vibrating offset
            self.canvas.create_image(cx + offset, cy + offset, image=self.dj_img,
                                    anchor="center", tags="visual")
        else:
            # Standard Idle State
            self.canvas.configure(bg=self.BG_COLOR)
            if self.dj_img and cx > 1:
                self.canvas.create_image(cx, cy, image=self.dj_img,
                                        anchor="center", tags="visual")

    # Refresh animation every 100ms
    self.after(100, self.update_ui_animation)
```

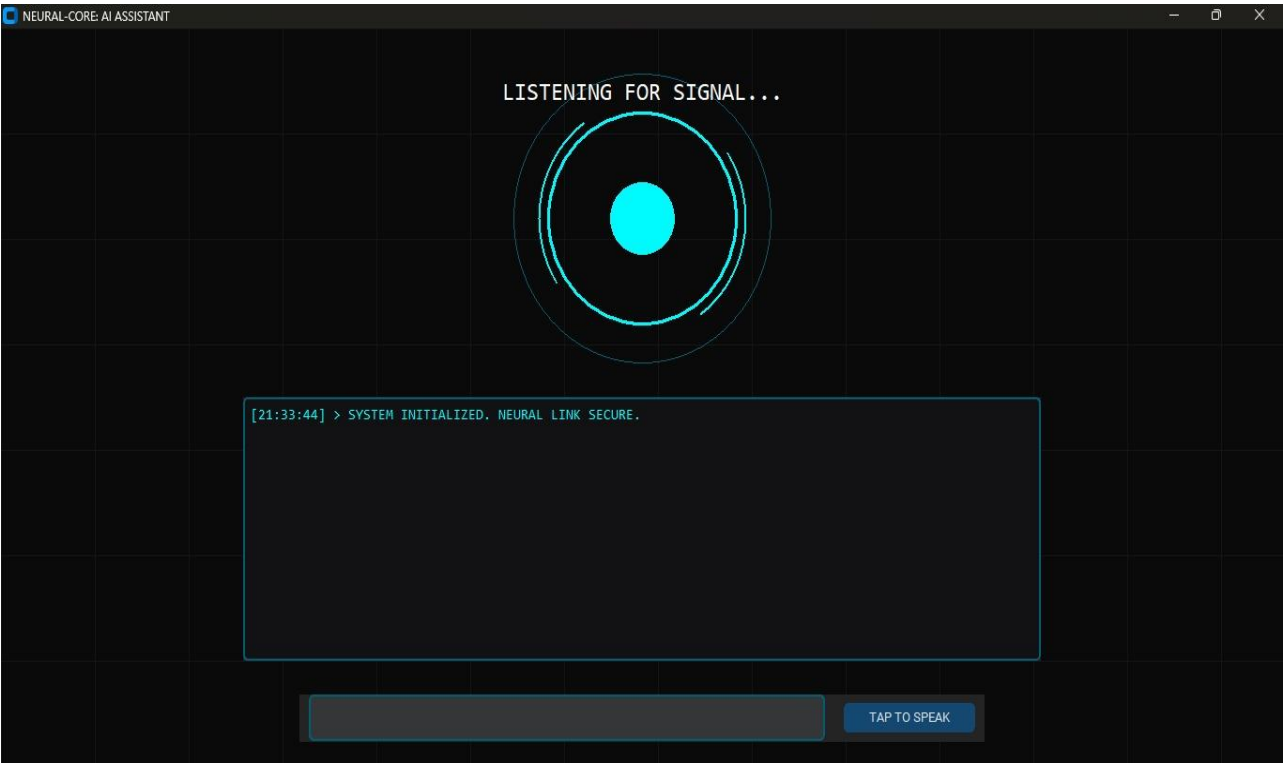
## 9.2 APPENDIX B

### SCREENSHOT

#### LOGINPAGE:

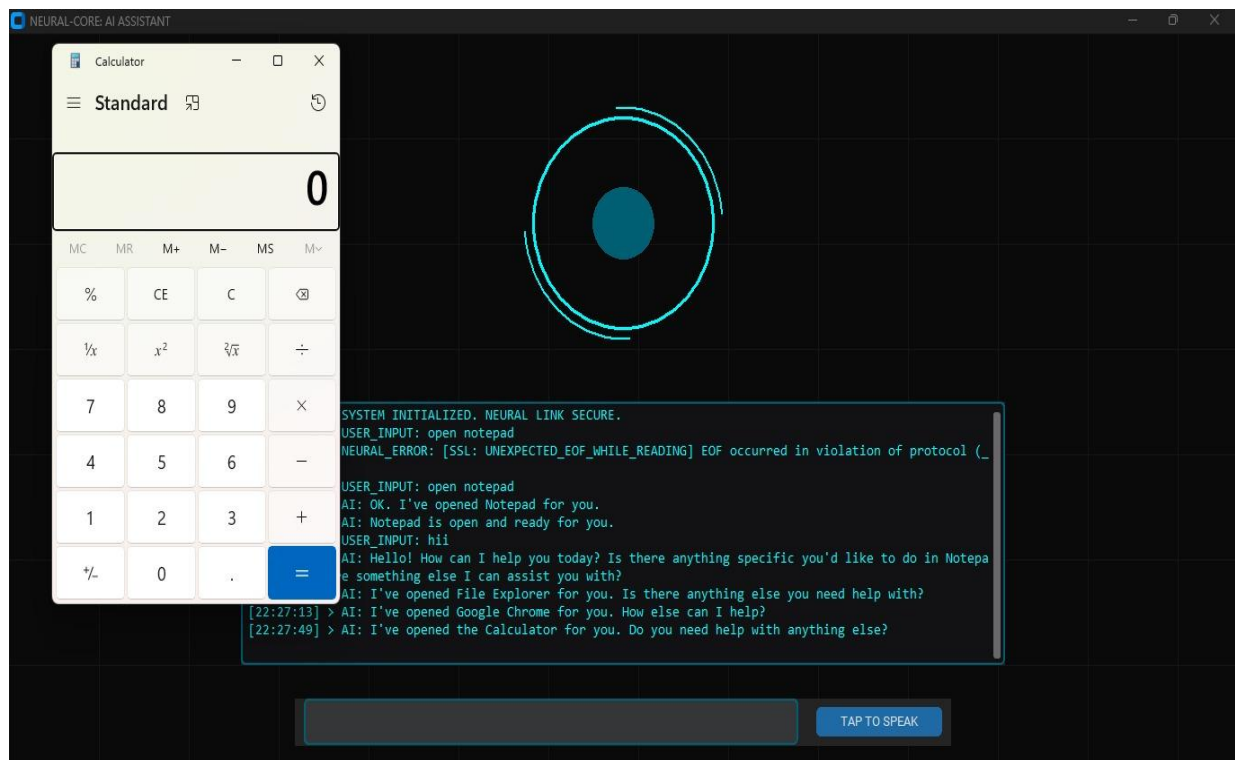


**HOMEPAGE:**



**OUTPUT:**

**COMMAND TO BE OPENED**



## SEARCHING:

